

Tema: Sistema de Controle de Adoção de Pets

Grupo: Ana Clara T. Ronzani, João Vitor Schmitt e Leonardo Victor Müller

Processo de Software

Processo escolhido: Scrum (Ágil)

Foi escolhido o Scrum por permitir organizar o desenvolvimento em pequenas etapas, facilitando a divisão das tarefas entre os integrantes do grupo.

Organização da equipe:

Integrante	Responsabilidade
Ana	Levantamento dos requisitos
Leonardo	Modelagem (diagrama UML)
João	Implementação
Todos	Testes e documentação

Etapas realizadas

1. Levantamento dos requisitos.
2. Modelagem do sistema.
3. Implementação das funcionalidades.
4. Testes unitários.
5. Documentação.

Requisitos de Software

Requisitos Funcionais:

- Cadastrar um pet.
- Listar os pets disponíveis para adoção.
- Cadastrar um adotante.
- Registrar a adoção de um pet.
- Consultar os pets já adotados.

Requisitos Não Funcionais:

- O sistema deve possuir menu simples.

- O código deve seguir orientação a objetos.
- O sistema deve permitir manutenção futura.

Histórias de Usuário:

- Como funcionário da ONG quero cadastrar um pet, para disponibilizá-lo para adoção.
- Como funcionário da ONG quero cadastrar um adotante, para registrar quem adotou o animal.
- Como funcionário quero registrar uma adoção, para remover o pet da lista de disponíveis.
- Como funcionário quero consultar os pets disponíveis, para informar aos interessados.

Modelo de Software

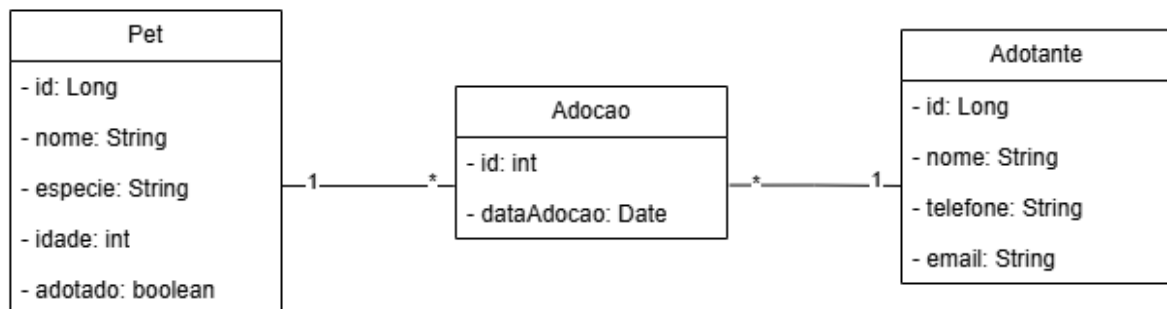
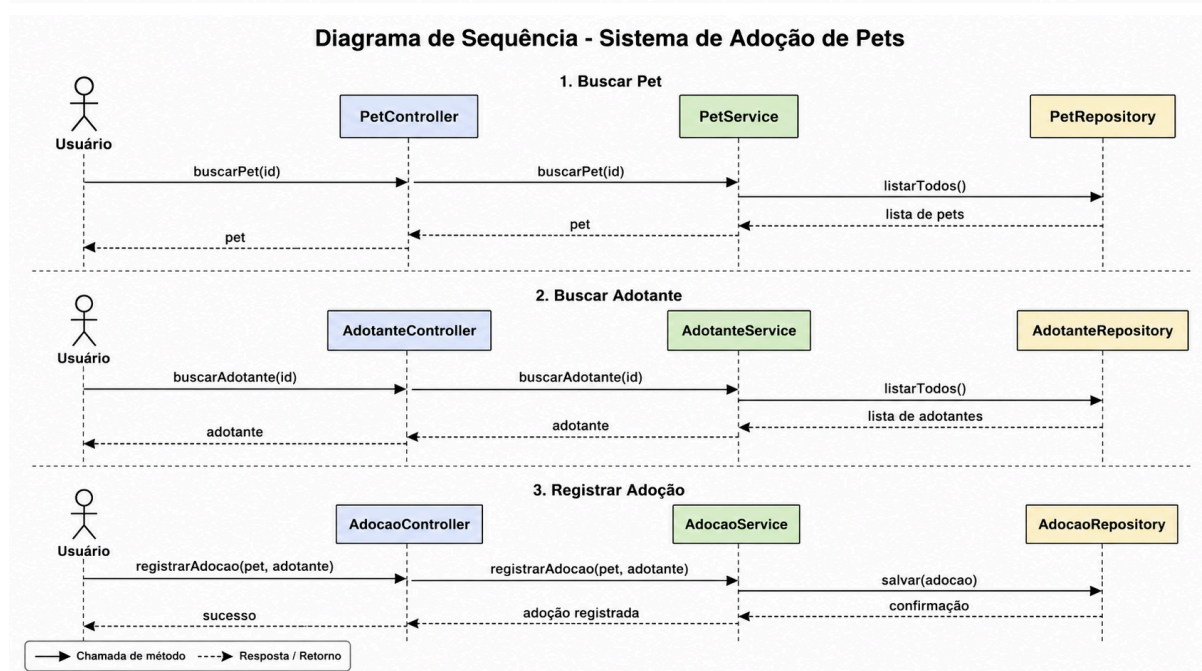
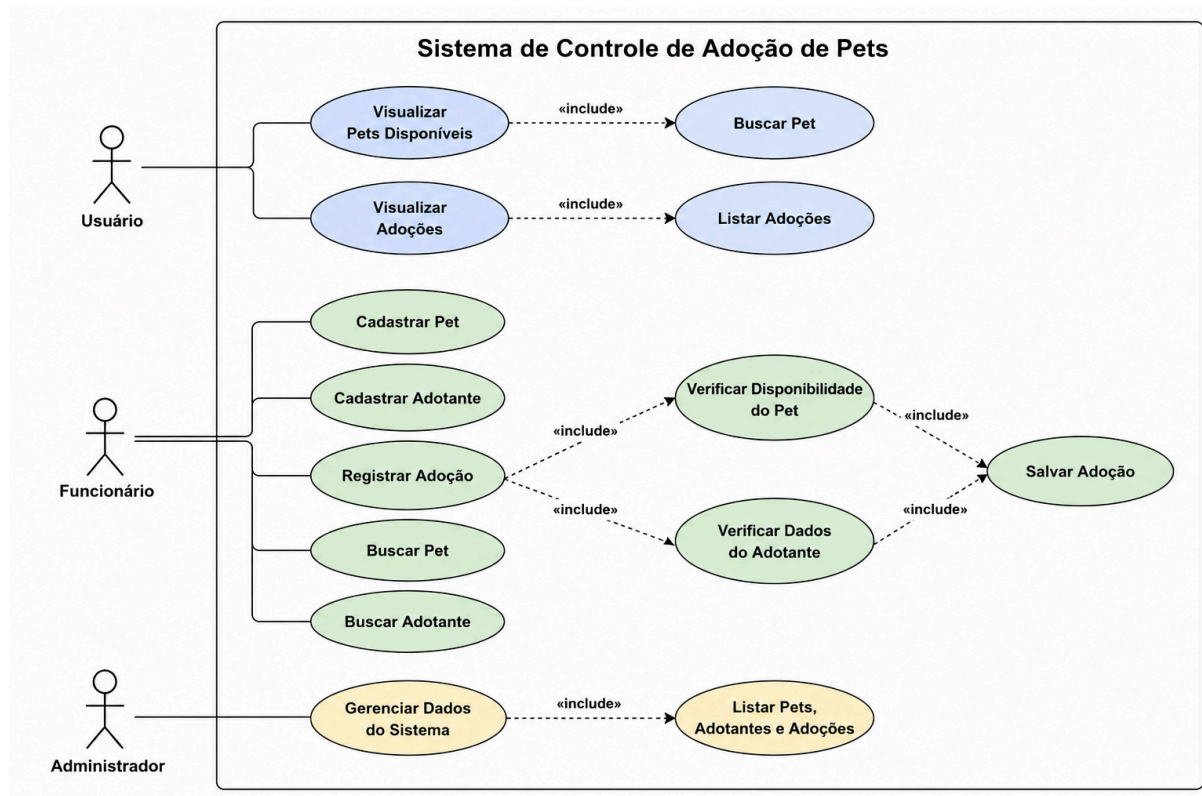


Diagrama de casos de uso



Princípios e Propriedades de Projeto

Integridade Conceitual:

Todas as classes representam elementos desse domínio, como Pet, Adotante e Adocao, mantendo uma organização consistente e uma arquitetura uniforme baseada nas camadas de Controller, Service e Repository.

Coesão:

Cada classe possui uma responsabilidade bem definida.

- As classes Pet, Adotante e Adocao representam apenas os dados do sistema.
- Os Controllers são responsáveis por receber as solicitações da interface e encaminhá-las para os serviços.
- Os Services implementam as regras de negócio.
- Os Repositories realizam o armazenamento e recuperação dos dados em memória.

Essa divisão aumenta a organização e facilita futuras alterações no sistema.

Baixo Acoplamento:

O sistema utiliza interfaces para os repositórios, fazendo com que a camada de serviços dependa de abstrações em vez de implementações concretas. Dessa forma, caso seja necessário substituir o armazenamento em memória por um banco de dados, apenas a implementação do repositório precisará ser alterada, sem impacto na lógica de negócio.

Além disso, os controladores dependem apenas das classes de serviço, reduzindo a dependência direta entre as diferentes camadas do sistema.

Ocultação de Informação:

Os atributos das classes de modelo foram declarados como privados, sendo acessados apenas por meio de métodos getters e setters. Dessa forma, os detalhes internos dos objetos ficam protegidos e o acesso aos dados ocorre de maneira controlada.

Princípios SOLID

Single Responsibility Principle (SRP):

Cada classe possui apenas uma responsabilidade específica. Os modelos representam entidades do sistema, os serviços implementam as regras de negócio, os repositórios realizam o armazenamento dos dados e os controladores fazem a comunicação entre a interface e os serviços.

Open/Closed Principle (OCP):

A utilização de interfaces para os repositórios permite adicionar novas implementações sem modificar a lógica dos serviços. Por exemplo, o repositório em memória pode ser substituído por um repositório utilizando banco de dados mantendo a mesma interface.

Liskov Substitution Principle (LSP):

As implementações dos repositórios podem ser utilizadas no lugar de suas interfaces sem alterar o comportamento esperado pelos serviços.

Interface Segregation Principle (ISP):

As interfaces dos repositórios contêm apenas os métodos necessários para suas operações, evitando que implementações sejam obrigadas a fornecer funcionalidades que não utilizam.

Dependency Inversion Principle (DIP):

Os serviços dependem das interfaces dos repositórios e não das implementações concretas. Essa abordagem facilita a realização de testes unitários utilizando mocks e torna o sistema mais flexível.

Testes de Software

Para garantir o correto funcionamento do sistema de controle de adoção de pets, foram implementados testes unitários utilizando JUnit 5 e Mockito. Os testes foram desenvolvidos com o objetivo de validar as regras de negócio de forma isolada, simulando as dependências por meio de objetos mock.

Testes Unitários

Os testes unitários foram realizados sobre a camada de serviços (Service), responsável pela implementação das regras de negócio do sistema.

PetService

Foram implementados testes para validar as seguintes funcionalidades:

- Cadastro de um novo pet, verificando se o repositório recebe corretamente a solicitação de armazenamento.
- Listagem de pets disponíveis para adoção, garantindo que apenas animais não adotados sejam retornados.

AdotanteService

Os testes realizados validam:

- Cadastro de um novo adotante.
- Listagem de todos os adotantes cadastrados.
- Busca de um adotante pelo seu identificador.

AdocaoService

Foram implementados testes para verificar:

- Registro de uma nova adoção.
- Listagem das adoções cadastradas.
- Impedimento da adoção de um pet que já foi adotado anteriormente, garantindo a aplicação correta da regra de negócio.

Uso de Mocks

Durante os testes foi utilizado o framework Mockito para criar objetos simulados (mocks) dos repositórios. Dessa forma, foi possível testar apenas a lógica de negócio presente na camada de serviços, sem depender da implementação real do armazenamento em memória.